# Мониторинг, логирование и диагностика Docker

Состояния, журналы, ресурсы и устранение ошибок в многоконтейнерном приложении

# Цель лекции

- Различать проверку состояния, логов и ресурсов Docker-контейнеров.
- Применять диагностику к Nginx → Express API → PostgreSQL.
- Системно находить и устранять типичные неисправности без потери данных.

# Учебные вопросы

| № | Учебный вопрос |
|---|---|
| 1 | Состояния контейнеров и Compose-сервисов |
| 2 | Логи контейнеров и приложение |
| 3 | Мониторинг ресурсов контейнеров |
| 4 | Диагностика конфигурации и сети |
| 5 | Системный алгоритм поиска и устранения ошибок |

# Исходная учебная система

- Три сервиса Compose: nginx принимает запросы; api обрабатывает маршруты; db хранит PostgreSQL-данные.
- Внутри сети сервисы используют имена api и db; с хоста HTTP доступен через порт Nginx.
- Данные PostgreSQL сохраняются в именованном Docker Volume.
- Исследуем эту систему, а не переписываем код приложения.

# Что означают три категории наблюдений

### Состояние

- Контейнер запущен или остановлен?
- Какой код завершения?
- Есть ли перезапуски?

### Поведение и ресурсы

- Что приложение пишет в stdout/stderr?
- Есть ли ответы на HTTP-запрос?
- Сколько CPU и памяти используется?

# Состояния контейнеров и Compose-сервисов

Сначала выясняем, работает ли процесс и какой сервис затронут

# Схема: Диагностика Docker — от состояния к причине

Пошаговый путь поиска неисправности в контейнере и сервисах Docker Compose

## 1 Симптом

Проблема проявляется на уровне пользователя или в сервисе.

- Сайт не открывается
- API не отвечает
- Ошибка БД

## 2 Состояние контейнеров

Сначала проверяем, какие контейнеры запущены.

```
$ docker ps
$ docker compose ps
```

- **running** — контейнер запущен
- **exited** — контейнер остановлен
- **restarting** — контейнер перезапускается

## 3 Логи

Смотрим логи контейнера или сервиса, чтобы найти ошибку.

```
$ docker logs <container>
$ docker compose logs api
```

```
Connection refused
port already in use
ECONNREFFUSED db:5432
```

В логах обычно есть точная причина сбоя.

## 4 Конфигурация

Проверяем переменные, порты и итоговую конфигурацию.

```
$ docker inspect <container>
$ docker compose config
$ printenv
```

- **Порт** — настройки портов (ports)
- **DB_HOST** — адрес БД (env)
- **Volume** — подключённые тома
- **Env** — переменные окружения

## 5 Сеть и ресурсы

Проверяем соединение сервисов и нагрузку на ресурсы.

```
$ docker exec api ping db
$ docker stats
$ docker top <container>
```

- **Сеть** — доступность сервисов (ping, DNS, порты)
- **CPU** — загрузка процессора
- **RAM** — использование памяти

## 6 Причина и действие

| Типовые причины | Действия |
|---|---|
| **Неверный DB_HOST** — контейнер не может найти БД | **Исправить compose.yaml** — укаазать правильный DB_HOST |
| **Контейнер остановлен** — сервис не запущен | **Перезапустить сервис** — docker compose restart <service> |
| **Ошибка порта** — порт уже занят или не проброшен | **Проверить логи** — docker logs ... смотреть сообщение об ошибке |
| **Проблема сети** — сервисы не видят друг друга | **Восстановить сеть** — проверить настройки сети, при необходимости создать заново |
| **Неверная переменная окружения** — ошибочные значения в .env или compose.yaml | **Пересоздать контейнер без удаления тома** — docker compose up -d --force-recreate |

## Что важно понять

**Идём от симптома к фактам**

Не гадаем, а последовательно собираем доказательства командами.

**Сначала состояние, потом логи**

Сначала проверяем, какие контейнеры запущены, затем анализируем логи.

**Конфигурацию проверяем отдельно от кода**

Проблема может быть в окружении, портах, томах и переменных, а не в коде приложения.

**Причина должна подтверждаться командой**

Любой вывод или догадка должны быть подтверждены реальной командой Docker.

## Мини-алгоритм

1. `ps` — проверяем состояние контейнеров
2. `logs` — смотрим логи и ищем ошибку
3. `inspect/config` — проверяем конфигурацию
4. `network/stats` — проверяем сеть и нагрузку
5. `fix` — устраняем причину

# Состояния контейнера

- created — контейнер существует, но основной процесс не запущен.
- running — основной процесс выполняется; это не доказывает, что API отвечает корректно.
- exited — основной процесс завершился; важны ExitCode и логи.
- restarting / paused — дополнительные состояния, требующие проверки причин.

# Проверка контейнеров и сервисов

BASH

```
docker ps -a
docker compose ps -a
docker compose ls
```

**Что важно**

- docker ps -a показывает все контейнеры Docker Engine.
- docker compose ps -a ограничивает обзор текущим Compose-проектом; запускайте из каталога проекта.
- docker compose ls показывает известные Compose-проекты.

# Docker inspect: состояние и причина остановки

```bash
docker inspect --format '{{.State.Status}} {{.State.
```

**Что важно**

- State.Status — текущее состояние.
- State.ExitCode — код завершения основного процесса; 0 обычно означает штатное завершение.
- RestartCount помогает заметить повторные запуски, но сам по себе не объясняет причину.

# Контейнер running — не значит приложение healthy

- Процесс может работать, а соединение с БД — не устанавливаться.
- Проверьте HTTP-ответ и бизнес-функцию, например GET /api/products.
- HEALTHCHECK полезен для проверки работоспособности, но его наличие не гарантировано.
- Статус health доступен только если проверка здоровья задана.

# Проверка на разных уровнях

| Уровень | Инструмент | Что узнаём |
| --- | --- | --- |
| Контейнер | docker ps / inspect | Основной процесс и код завершения |
| Compose | docker compose ps | Статус сервиса |
| HTTP | curl -i ... | Код ответа, путь и доступность |
| База | psql / SQL-запрос | Наличие данных и работа запросов |

# Логи Docker и приложений

Логи — первое место для поиска причин ошибок

# Схема: Потоки stdout/stderr → Docker logs → администратор

Как сообщения приложении попадают в журналы Docker и используются для диагностики

## Приложение в контейнере




```
console.log( 'Server started...')
[INFO] Server started on port 3000

console.error('DB connection failed')
[ERROR] ECONNREFUSED db:5432
```

## Docker container runtime

**Основной процесс контейнера**

```
> node app.js
```

поток вывода (stdout)

поток ошибок (stderr)

**Docker собирает stdout и stderr** основного процесса

## Docker logs / Docker Compose logs

```
$ docker logs api
  # показать логи контейнера
$ docker compose logs api
  # показать логи сервиса
$ docker compose logs -f api
  # смотреть логи в реальном времени
```

- Просмотр логов контейнера или сервиса
- Просмотр в реальном времени (-f)
- Фильтрация по сервису
- Использование для диагностики проблем

## Администратор




- **Ищет ошибку** анализирует сообщения
- **Проверяет запуск** убеждается, что сервис работает
- **Диагностирует проблему** находит причину сбоя

## Что важно понять

1. `console.log()` обычно попадает в `stdout`.
2. Ошибки и аварийные сообщения часто идут в `stderr`.
3. Docker собирает оба потока и показывает их через `docker logs`.
4. В Compose можно смотреть логи отдельного сервиса.
5. Логи помогают найти причину сбоя без входа в код.

## Типичные примеры

```
[INFO] Server started on port 3000
[INFO] Connected to PostgreSQL
[ERROR] ECONNREFUSED db:5432
```

# Источники журналов

- stdout — стандартный вывод приложения (например, console.log).
- stderr — поток сообщений об ошибках (например, console.error).
- docker logs показывает логи, доступные через настроенный logging driver; состав зависит от драйвера.
- Не все файлы логов, записанные внутрь контейнера, автоматически попадают в docker logs.

# Основные варианты docker logs

```bash
docker logs CONTAINER_NAME
docker logs --tail 30 CONTAINER_NAME
docker logs -f CONTAINER_NAME
docker logs --since 10m --timestamps CONTAINER_NAME
```

**Что важно**

- --tail ограничивает объём выдачи.
- -f следит за новыми сообщениями (выход Ctrl+C).
- --since и --timestamps удобны для сопоставления события с ошибкой.

# Логи конкретного сервиса Compose

BASH

```
docker compose logs --tail 30 api
docker compose logs -f nginx
docker compose logs --since 5m --timestamps db
```

**Что важно**

- Имя api — имя сервиса в compose.yaml, а не произвольное имя контейнера.
- Можно анализировать каждый сервис отдельно либо использовать docker compose logs без имени.

# Как читать ошибки приложений

- ECONNREFUSED — соединение отвергнуто; проверить адрес, порт и готовность сервиса.
- ENOTFOUND — имя хоста не разрешено; проверить имя сервиса, сеть и переменные окружения.
- HTTP 502 у Nginx — проверить доступность upstream API и логи Nginx.
- Не путать симптом (502) с первопричиной (например, контейнер API завершился).

# Корреляция HTTP-запроса с логами

1. **Шаг 1** — Выполните curl к Nginx и запомните время запроса.
2. **Шаг 2** — Проверьте HTTP-статус и тело ответа.
3. **Шаг 3** — Посмотрите docker compose logs nginx, затем api.
4. **Шаг 4** — При подозрении на БД проверьте db и конфигурацию DB_HOST.
5. **Шаг 5** — Сформулируйте первопричину и способ восстановления.

# Мониторинг ресурсов

Что происходит с CPU, памятью, вводом-выводом и процессами

# Схема: docker stats — CPU, память, сеть и процессы

Как администратор контролирует использование ресурсов контейнеров

## Контейнеры




## docker stats

Сбор и отображение статистики контейнеров



## Администратор

Анализирует использование ресурсов и выявляет проблемы

## Команда мониторинга

```
$ docker stats
$ docker stats api db nginx
$ docker stats --no-stream
```

Показывает использование ресурсов контейнеров в реальном времени.

## Что показывает docker stats

**CPU %**
Загрузка процессора контейнера.

**MEM USAGE / LIMIT**
Использование и лимит памяти контейнера.

**NET I/O**
Входящий и исходящий сетевой трафик.

**PIDS / PROCESSES**
Количество процессов в контейнере.

## Пример вывода

```
$ docker stats
```

| CONTAINER | CPU % | MEM USAGE / LIMIT | NET I/O | PIDS |
|---|---|---|---|---|
| nginx | 0.15% | 18MiB / 512MiB | 120kB / 95kB | 2 |
| api | 1.80% | 95MiB / 512MiB | 540kB / 420kB | 12 |
| db | 3.10% | 180MiB / 1GiB | 1.2MB / 980kB | 9 |

## Что важно понять

**Высокий CPU**
может указывать на нагрузку или цикл обработки.

**Рост памяти**
помогает заметить утечки памяти.

**NET I/O**
показывает активность обмена данными.

**Число процессов**
помогает увидеть аномальное поведение контейнера.

## Правильно

- Сравнивать контейнеры между собой
- Проверять stats вместе с logs
- Использовать `--no-stream` для разовой проверки

## Неправильно

- Судить о проблеме только по одной метрике
- Игнорировать лимиты памяти
- Не сопоставлять метрики с логами

# Наблюдение за ресурсами

BASH

```
docker stats
docker stats --no-stream
docker compose stats --no-stream
```

**Что важно**

- docker stats обычно показывает поток текущих метрик; остановка Ctrl+C.
- --no-stream выводит один снимок.
- Не делайте вывод о проблеме по одному измерению без базового уровня.

# Как читать docker stats

| Поле | Значение | Практический вывод |
| --- | --- | --- |
| CPU % | Использование процессора | Повышение может указывать на нагрузку |
| MEM USAGE / LIMIT | Память и лимит | Сопоставить с установленным ограничением |
| NET I/O | Обмен по сети | Проверка активности трафика |
| BLOCK I/O | Дисковый ввод/вывод | Нагрузка на хранение |
| PIDS | Число процессов/потоков | Необычный рост требует анализа |

# Мониторинг — не равен нагрузочному тесту

- docker stats собирает текущую статистику, а не доказывает производительность под нагрузкой.
- Показатель CPU% зависит от способа расчёта и числа процессоров.
- Один снимок не позволяет установить тренд.
- Причины высокой нагрузки требуют изучения запросов, логов и поведения приложения.

# Процессы внутри сервиса

BASH

```
docker compose top
docker compose exec api ps -ef
```

**Что важно**

- docker compose top показывает процессы сервисов.
- ps -ef внутри контейнера сработает, только если утилита ps установлена в образе.
- Для минимальных образов используйте docker top или docker compose top.

# Диагностика конфигурации и сети

Проверяем переменные, DNS, доступность портов и корректность YAML

# Схема: Диагностика Nginx → API → PostgreSQL по уровням

Как искать причину сбоя по уровням приложения: веб-сервер, backend и база данных




## 1. Уровень Nginx

**Симптомы:**
502 Bad Gateway, 404,
сайт не открывается

**Полезные команды:**

```
$ docker compose logs nginx
$ docker compose exec nginx nginx -t
$ curl http://localhost:8080/api/products
```

**Что проверить:**

- порт опубликован (например, 8080:80)
- конфиг nginx корректный
- proxy_pass указывает на api:3000

Если Nginx не может достучаться до API — проблема выше по цепочке.

## 2. Уровень API

node + Express

**Симптомы:**
500 Internal Server Error,
API не отвечает,
JSON не загружается

**Полезные команды:**

```
$ docker compose logs api
$ docker compose exec api printenv DB_HOST
$ docker compose exec api ping db
```

**Что проверить:**

- контейнер api запущен
- переменные окружения верны
- маршруты /api работают
- приложение слушает порт 3000

**Примеры ошибок:**

```
ECONNREFUSED db:5432
Cannot GET /api/products
```

## 3. Уровень PostgreSQL

PostgreSQL

**Симптомы:**
ошибка подключения к БД,
SQL-запросы не выполняются

**Полезные команды:**

```
$ docker compose logs db
$ docker compose exec db pg_isready -U shopuser
$ docker compose exec api nc -zv db 5432
```

**Что проверить:**

- сервис db запущен
- порт 5432 доступен внутри сети
- POSTGRES_DB / USER / PASSWORD заданы
- том с данными подключён

**Примеры (проверка из psql):**

```
database 'shop' exists
table products
```

## Как идти по уровням

1. **Проверить Nginx** — Убедиться, что запрос доходит до backend
2. **Проверить API** — Убедиться, что API работает и может подключиться к БД
3. **Проверить PostgreSQL** — Убедиться, что БД доступна и отвечает на запросы
4. **Подтвердить причину и исправить** — На основе найденной ошибки внести изменения

### Правильно

- идти от внешнего уровня к внутреннему
- смотреть логи каждого сервиса
- проверять сеть и env

### Неправильно

- сразу менять код без диапностики
- искать ошибку только в одном сервисе
- игнорировать конфигурацию Compose

# Проверка конфигурации и окружения

BASH

```
docker compose config -q
docker compose ps
docker compose exec api printenv DB_HOST
docker compose exec api printenv DB_PORT
```

**Что важно**

- config -q проверяет конфигурацию без её печати.
- printenv показывает переменную конкретного контейнера; выводите только несекретные значения.
- Параметр DB_HOST должен указывать на сервис db, а не на localhost.

# Проверяем API с точки зрения клиента

BASH

```
curl -i http://127.0.0.1:8080/api/products
docker compose logs --tail 30 nginx
docker compose logs --tail 30 api
```

**Что важно**

- curl -i показывает HTTP-статус и заголовки.
- Если HTTP-порт изменён в лекции №9, замените 8080 на текущий.
- 502 от Nginx требует проверки API и доступности upstream.

# DNS в Compose: где искать ошибку

- По умолчанию сервисы одного Compose-проекта общаются по именам сервисов.
- DB_HOST=db означает обращение к сервису db внутри Compose-сети.
- localhost внутри контейнера api — это сам api, а не PostgreSQL.
- Nginx обращается к upstream http://api:3000; клиент обращается к опубликованному HTTP-порту хоста.

# Увидеть сеть и тома

BASH

```
docker network ls
docker compose config --services
docker volume ls
docker compose ps -a
```

**Что важно**

- config --services выводит имена сервисов конфигурации.
- Для проверки существования хранилища используйте volume ls.
- Не применяйте down -v при диагностике, чтобы не уничтожить учебные данные.

# Алгоритм устранения неисправностей

От симптома к воспроизводимой проверке и восстановлению

# Схема: Алгоритм поиска неисправности Docker Compose

## Пошаговый порядок диагностики проблем в многоконтейнерном приложении

### 1 Симптом

Сайт не работает или ведет себя неправильно



- сайт не открывается
- API не отвечает
- ошибка БД
- страница 500
- медленная работа

### 2 Проверить состояние сервисов

Проверим, запущены ли контейнеры и в каком они состоянии.

```
$ docker compose ps
$ docker ps
```

**Возможные статусы:**

- `running` — работает
- `exited` — остановлен
- `restarting` — перезапускается

Проверьте все сервисы: nginx, api, db и др.

### 3 Проверить логи

В логах часто сразу видно ошибки и причину неисправности.

```
$ docker compose logs
$ docker compose logs api
$ docker logs <container>
```




- ошибки приложения
- ошибки подключения к БД
- исключения в коде
- проблемы конфигурации

### 4 Проверить конфигурацию

Убедимся, что конфигурация корректна и соответствует ожиданиям.

```
$ docker compose config
$ docker inspect <container>
$ printenv
```

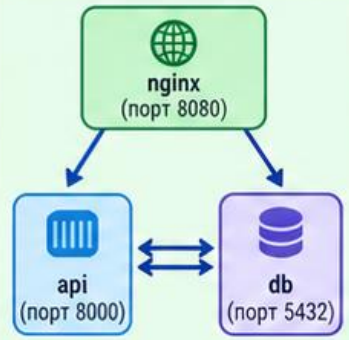



- порты
- тома (volumes)
- переменные окружения
- имена сервисов
- зависимости между сервисами

### 5 Проверить сеть и доступность

Проверим, могут ли сервисы взаимодействовать друг с другом.

```
$ docker compose exec api ping db
$ docker compose exec api nc -zv db 5432
$ curl http://localhost:8080/api/products
```




Убедитесь, что nginx, api и db могут соединяться между собой.

### 6 Проверить ресурсы

Исключим проблемы с ресурсами хоста или контейнеров.

```
$ docker stats
$ docker top <container>
```



- использование CPU
- использование RAM
- нагрузка на контейнеры
- процессы внутри контейнера
- возможные утечки памяти

### 7 Найти причину

На основе собранных фактов определим корень проблемы.



- неверный DB_HOST
- остановлен контейнер
- неправильно проброшен порт
- ошибка в синтаксисе YAML
- приложение упало
- проблемы с сетью
- неверные переменные окружения

### 8 Исправить и проверить повторно

Внесем изменения и убедимся, что проблема решена.



- исправить compose.yaml или .env
- перезапустить сервис (docker compose restart)
- при необходимости пересоздать контейнер (docker compose up -d)
- снова проверить работу приложения

## Мини-алгоритм

Краткий порядок действий:

1. ps
2. logs
3. config / inspect
4. network
5. stats
6. fix
7. recheck

## Правильно

- идти от симптома к фактам
- сначала состояние и логи
- подтверждать причину командой
- проверять после исправления

## Неправильно

- сразу менять код
- игнорировать логи
- удалять том без необходимости
- делать выводы без проверки

# Универсальный порядок диагностики

| | | |
|---|---|---|
| 1 | **Шаг 1** | Зафиксировать симптом и команду, которая его воспроизводит. |
| 2 | **Шаг 2** | Проверить docker compose ps -a и коды завершения. |
| 3 | **Шаг 3** | Посмотреть логи неисправного сервиса (tail/since). |
| 4 | **Шаг 4** | Проверить compose config и несекретные переменные окружения. |
| 5 | **Шаг 5** | Проверить сеть, DNS-имена и порты, затем БД. |
| 6 | **Шаг 6** | Исправить одну причину и повторить исходную проверку. |

# Симптом → проверка → решение

| Симптом | Где проверять | Возможное действие |
|---|---|---|
| Контейнер exited | ps -a, inspect, logs | Исправить ошибку запуска; up -d |
| 502 Bad Gateway | nginx logs, api ps/logs | Восстановить API / upstream |
| Не найден db | DB_HOST, сеть, logs | Вернуть DB_HOST=db |
| Нет HTTP-ответа | ports, curl, nginx logs | Исправить публикацию порта |
| Не хватает памяти | stats, limits, logs | Измерить нагрузку и скорректировать лимит |

# Безопасное восстановление Compose-проекта

```bash
docker compose config -q
docker compose up -d
docker compose ps
curl -i http://127.0.0.1:8080/api/products
```

**Что важно**

- up -d создаёт/запускает сервисы и применяет изменения, когда необходимо.
- Данные именованного тома PostgreSQL сохраняются, если не удалять том.
- После восстановления снова проверьте исходный симптом.

# Что фиксировать в отчёте об инциденте

- Описание симптома: HTTP-статус, время, затронутый сервис.
- Доказательства: команды, сокращённые логи, состояния и метрики.
- Первопричина: конкретная неверная настройка или остановившийся процесс.
- Исправление и повторная проверка: до / после.
- Не включайте в скриншоты и отчёты реальные пароли и токены.

# Контрольные вопросы

1. Почему статус running не гарантирует успешный ответ API?
2. Чем отличаются docker logs и docker compose logs?
3. Для чего нужны --tail и --since?
4. Какие поля выводит docker stats?
5. Почему DB_HOST=localhost неверен для отдельного контейнера БД?
6. Как отличить HTTP 502 от ошибки PostgreSQL?
7. Почему нельзя использовать down -v при обычной диагностике?
8. Что следует проверять после исправления ошибки?

# Итоги лекции

1. Проверяйте состояние: ps, inspect, Compose ps.
2. Читайте журналы: logs и compose logs, tail/follow/since.
3. Оценивайте ресурсы: stats и top; отличайте наблюдение от тестирования.
4. Проверяйте конфигурацию, сеть, DNS и HTTP последовательно.
5. Восстанавливайте сервисы без удаления томов с данными.